

# Manual de Python

Univ. Rafael Villca Poggian

2018

# Índice general

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1. El Lenguaje</b>                         | <b>3</b>  |
| 1.1. Introducción al Lenguaje . . . . .       | 3         |
| 1.1.1. ¿Por qué Python? . . . . .             | 3         |
| 1.1.2. Cómo Instalarlo . . . . .              | 3         |
| 1.1.3. Editor Recomendado . . . . .           | 3         |
| 1.2. Lectura y Escritura . . . . .            | 4         |
| 1.2.1. Salida . . . . .                       | 4         |
| 1.2.2. Entrada . . . . .                      | 4         |
| 1.2.3. Castear Tipos de Datos . . . . .       | 5         |
| 1.3. Estructuras Condicionales . . . . .      | 5         |
| 1.3.1. If . . . . .                           | 5         |
| 1.3.2. Else . . . . .                         | 5         |
| 1.3.3. Elif . . . . .                         | 6         |
| 1.3.4. Condicional Ternaria . . . . .         | 6         |
| 1.4. Estructuras Repetitivas . . . . .        | 6         |
| 1.4.1. While . . . . .                        | 7         |
| 1.4.2. Range . . . . .                        | 7         |
| 1.4.3. For . . . . .                          | 7         |
| 1.5. Funciones . . . . .                      | 7         |
| 1.5.1. Argumentos . . . . .                   | 8         |
| 1.5.2. Retorno . . . . .                      | 8         |
| 1.5.3. Funciones Lambda . . . . .             | 8         |
| 1.6. Estructuras . . . . .                    | 8         |
| 1.6.1. La Lista de Python . . . . .           | 8         |
| 1.6.2. Tuplas . . . . .                       | 10        |
| 1.7. Módulos y Librerías . . . . .            | 10        |
| <b>2. Aplicaciones en la Materia</b>          | <b>12</b> |
| 2.1. Primer Parcial . . . . .                 | 12        |
| 2.1.1. Algoritmos Secuenciales . . . . .      | 12        |
| 2.1.2. Estructuras Condicionales . . . . .    | 13        |
| 2.1.3. Series . . . . .                       | 14        |
| 2.1.4. Sumatorias . . . . .                   | 15        |
| 2.1.5. Lotes . . . . .                        | 16        |
| 2.1.6. Composición y Descomposición . . . . . | 16        |
| 2.2. Segundo Parcial . . . . .                | 16        |
| 2.2.1. Funciones y Procedimientos . . . . .   | 16        |
| 2.2.2. Vectores . . . . .                     | 18        |
| 2.2.3. Cadenas . . . . .                      | 20        |
| 2.2.4. Recursividad . . . . .                 | 21        |
| 2.3. Final . . . . .                          | 22        |
| 2.3.1. Matrices . . . . .                     | 22        |



# Capítulo 1

## El Lenguaje

### 1.1. Introducción al Lenguaje

#### 1.1.1. ¿Por qué Python?

Python es un lenguaje de tipado dinámico pensado para prototipar código rápidamente, esto y su gran comunidad, lo llevó a ser ampliamente usado para proyectos de desarrollo Web con módulos como Django, para desarrollo móvil mediante Kivi, desarrollo de aplicaciones de escritorio, y para lo que más se usa actualmente, aplicaciones de Machine Learning gracias a la facilidad de uso de sus librerías.

El fuerte de Python es la gran cantidad de librerías que tiene disponibles, además que las que ya tiene por defecto son muy poderosas, permitiendo escribir códigos en muy pocas líneas que en otros lenguajes tomaría muchas más para realizar la misma tarea.

Una desventaja de Python es su velocidad, al ser interpretado, su ejecución dura más de lo que un lenguaje compilado pudiera llegar a durar.

Por otra parte, otra desventaja es que al ser de tipado dinámico, para aplicaciones pequeñas puede ser útil, sin embargo, para proyectos muy grandes, uno puede perderse si no especifica en comentarios el tipo de dato que toman las variables importantes en todo el código.

#### 1.1.2. Cómo Instalarlo

La instalación más segura y completa de Python, la ofrece Anaconda, un gestor de paquetes para el lenguaje.

Para instalarla debemos acceder a la página de Anaconda, y descargar Anaconda, si se desea tener la mayoría de librerías, caso contrario se puede usar MiniConda.

#### 1.1.3. Editor Recomendado

Pycharm es uno de los mejores editores para Python, al tener un autocompletado muy eficiente para el lenguaje.

El otro editor de código, muy usado para prototipar código científico, **Jupyter Notebook**.

## 1.2. Lectura y Escritura

### 1.2.1. Salida

La función *print*, nos permite imprimir el contenido de una variable a la pantalla por consola.

Esta función nos permite modificar la forma de la salida que deseamos, siendo las dos estilizaciones más comunes la separación entre variables a imprimir, y el caracter de fin de línea.

```
>>> n = 12 #Asignamos un valor numérico a "n"
>>> print(n) #Mostramos el valor de "n"
12 #La salida de la función Print
>>> s = "hola mundo" #Asignamos una cadena de texto a "s"
>>> print(s)
hola mundo #Se muestra el contenido de la variable "s"
>>> s1 = 'hola mundo' #Es equivalente el uso de comilla simple o doble
>>> print(s1)
hola mundo
>>> s1
'hola mundo' #Si no usamos la función print, se muestra con comillas
>>> f = 1/3 #Igual podemos guardar decimales en variables
>>> print(f) #La función print igual puede mostrar el valor
0.3333333333333333
>>> s1 = 'hola'
>>> s2 = 'como'
>>> s3 = 'estas'
>>> print(s1, s2, s3) #Podemos imprimir varias variables a la vez
hola como estas #Se agrega un espacio automaticamente
>>> print(s1, s2, n) #Funciona con variables de diferentes tipos
hola como 12
>>> print(s1, s2, s3, sep='—') #Podemos cambiar el tipo de separación
hola—como—estas
>>> print(s1, s2, s3, sep='—', end='') #Podemos cambiar el caracter de fin de línea
hola—como—estas>>> #Para que no imprima un salto de línea
```

### 1.2.2. Entrada

La función *input*, es la forma estándar de lectura de datos por teclado, los datos que ingresemos se reciben como una cadena de texto dentro de la variable.

Podemos mostrar una línea de texto antes de recibir la entrada si la ingresamos como argumento de la función.

```
>>> var = input() #Le decimos al intérprete que lo ingresado se guardará en "var"
1998 #Ingresamos el dato por teclado
>>> print(var)
1998 #Verificamos que se ingresó correctamente
>>> var
'1998' #Al no formatear, observamos que es una cadena
>>> type(var)
<class 'str'> #Confirmamos que "var" contiene una cadena
>>> nombre = input('ingrese su nombre: ') #Cadena como parámetro
ingrese su nombre: Juan #Escribimos el nombre al lado de la cadena impresa
>>> print(nombre)
```

### 1.2.3. Castear Tipos de Datos

Dado que la entrada estándar siempre nos devolverá una cadena por teclado, muchas veces necesitaremos utilizar los datos como números enteros, o decimales. Para estas situaciones, debemos convertir el dato de tipo cadena a los tipos de datos que necesitemos utilizar, afortunadamente, Python ya tiene todas esas funciones pre implementadas.

```
>>> numero = int(input())      #La función int() convierte cadenas a enteros
33                             #Ingresamos el dato
>>> type(numero)
<class 'int'>                 #Verificamos que efectivamente "numero" contiene un entero
>>> numero
33                             #Al no tener comillas, observamos que es un número
>>> decimal = float(input())   #Usamos la función "float()"
3.56                           #Para convertir cadenas a decimales
>>> type(decimal)
<class 'float'>               #Verificamos que efectivamente "decimal" contiene un real
>>> decimal
3.56
```

## 1.3. Estructuras Condicionales

### 1.3.1. If

Es una estructura condicional, que recibe una expresión lógica que puede ser simple o compuesta y si se cumple, se ejecutan una serie de instrucciones.

En el caso de Python, a diferencia de otros lenguajes en los cuales las instrucciones a ejecutar dada la condición se engloban dentro de llaves, las líneas dentro de la condicional se definen en un nivel de indentación, para así expresar la jerarquía del código.

Para realizar comparaciones, se debe usar el operador `==` que es diferente al operador de asignación `=`.

```
>>> a = True                   #Asignamos el valor verdad a "a"
>>> if a == True:              #"if" verifica si la condición se cumple
...     print('hola')          #Se usa un bloque de indentación para indicar
...                             #que la instrucción se ejecuta bajo la condición
hola                           #Salida del programa
>>> b = False
>>> if a == True and b == False: #Se pueden usar expresiones compuesta
...     print('hola 2')
...
hola 2
```

### 1.3.2. Else

Es la contraparte de la condicional, permite definir un bloque de código a ejecutarse en caso de que la condición del `if` previo no se llegue a cumplir y se deba hacer algo en ese caso.

```
>>> a = True
>>> b = True
>>> if a == True and b == False:
...     print('hola')
... else:
...     print('adios')
...
adios
```

### 1.3.3. Elif

Es una variación del **Else** pero en este caso, permite al igual que el **if** normal usar una condicional, vendría a ser el equivalente lógico de decir “*Si no pasa esto, entonces, si es que pasa esto otro, hacer algo*”.

```
>>> a, b = False, True                                #Este tipo de asignación
                                                         #asigna: a = False y b = True
>>> if a == True and b == False:
...     print('hola')
... elif a == True and b == True:
...     print('adios')
... else:
...     print('bienvenido')
...
bienvenido
```

### 1.3.4. Condicional Ternaria

Nos permite escribir en una sola línea una condición **if-else**, sólo en el caso de que se tenga una instrucción para cada caso.

```
>>> a, b = True, False
>>> print( 'hola' if a == True and b == False else 'adios' )
hola
>>> a, b = True, True
>>> res = 'hola' if a == True and b == False else 'adios'
>>> print( res )                                     #Guardamos el resultado en una variable
adios
#Para este caso podemos ver que en realidad la condicional
#nos devuelve el texto para cada caso, y gracias a ello podemos
#meter la condicional dentro de la función print
```

## 1.4. Estructuras Repetitivas

Las estructuras repetitivas son estructuras de control, que nos permiten repetir un conjunto de instrucciones, en bucle, un número finito de veces, jugando con los valores que las variables pueden tomar dinámicamente.

### 1.4.1. While

Es un bucle que depende de una condicional que al igual que en la sentencia **If**, puede ser simple o compuesta. El conjunto de instrucciones se repetirá mientras la condicional se cumpla, en el momento en que ya no se dé el caso, el bucle finaliza y se procede a ejecutar la siguiente instrucción fuera del bucle.

```
>>> n = 0
>>> while n < 4:
...     print(n)
...     n = n + 1
...
0
1
2
3
```

### 1.4.2. Range

Es una función que permite generar un rango de valores de formas diferentes dependiendo de los argumentos que recibe. En el caso de que reciba un argumento  $n$ , el rango se genera desde 0 hasta  $n - 1$ . En el caso de que reciba dos argumentos  $n$  y  $m$ , el rango se genera desde  $n$  hasta  $m - 1$ .

### 1.4.3. For

Es un bucle en el que podemos definir un mayor control en las condiciones iniciales, siendo la variable iteradora, y el rango de valores en los que se iterará, mediante la función *range*.

```
>>> for i in range(2, 6):
...     print(i)
...
2
3
4
5
```

## 1.5. Funciones

Una función en el ámbito de la programación deriva del concepto de función matemática, como una relación que mapea un conjunto de  $n$  argumentos, y devuelve un conjunto de  $m$  elementos, siendo una relación  $\mathbb{R}^n \rightarrow \mathbb{R}^m$ .

En el caso del punto de vista computacional, una función recibirá argumentos, ejecutará un conjunto de instrucciones que pueden ser secuenciales, condicionales y repetitivas, para finalmente devolver un valor (o conjunto de valores) si se trata de una función pura, o puede no devolver nada, en cuyo caso se denomina **Procedimiento**.

En Python específicamente, se usa la palabra reservada **def**, para definir una función o procedimiento, y gracias a que es de tipado dinámico, el lenguaje inferirá solo, qué tipo de dato devuelve la función.



### 1.5.1. Argumentos

Es el conjunto de datos que recibirá la función y con los que trabajará localmente. Se definen en la cabecera de la función, entre paréntesis y luego del nombre de ésta.

```
def funcion(arg1, arg2, arg3, ...):  
    #Instrucciones
```

### 1.5.2. Retorno

Para devolver una o más variables al finalizar una función, debemos usar la instrucción *return*, al final de la ésta.

```
>>> def suma_rango(a, b):  
...     res = 0  
...     for i in range(a, b+1): #Iteramos hasta b+1 porque  
...         res = res + i       #Range corre hasta b-1  
...     return res             #Retornamos el resultado  
...  
>>> sum = suma_rango(1, 10)    #sum recibe la suma resultante  
>>> print(sum)  
55
```

### 1.5.3. Funciones Lambda

Son funciones que se pueden definir en una línea, que pueden o no tener nombre y tienen diversos usos por ejemplo al programar funciones matemáticas, o también al usar métodos de comparación complejos dentro de otras funciones.

```
>>> funcion = lambda x, y : (x+2) * y  
>>> print( funcion(2, 3) )  
12
```

## 1.6. Estructuras

### 1.6.1. La Lista de Python

La estructura llamada **lista** o **list** en Python, es en realidad un vector de tamaño dinámico.

#### Arreglo

Empezaremos por definir un Vector estático o Arreglo. Es una estructura de datos, que nos permite guardar nuestros elementos en casillas indexadas desde 0, de esta forma permitiéndonos acceder al i-ésimo elemento mediante su índice, para así poder trabajar con múltiples datos guardados de esta forma.

Un vector estático, tiene este nombre debido a que cuando se crea, se reserva en la memoria RAM, exactamente el espacio para n elementos de un tipo de dato específico y luego ya no se puede modificar el tamaño, quedándose con ese tamaño por siempre.

## Vector dinámico

Al igual que su contraparte estática, nos permite almacenar un conjunto de datos, sin embargo el funcionamiento de esta estructura es diferente, permite acceder a los elementos mediante el operador `[ ]`, pero además permite adicionar elementos dinámicamente al vector, de esta forma su tamaño puede crecer para ajustarse a la necesidad del programa, a parte de poder crearse ya con un tamaño definido o totalmente vacío.

En el caso de Python, su lista es un vector dinámico por defecto, el cual internamente utiliza una modificación de la estructura **vector<type>** de C++ implementada en C como una **Pylist**.

```
>>> Arr = []          #Creamos un vector vacío
>>> len(Arr)          #La función len() nos da la cantidad de elementos
0
>>> Arr.append(5)      #Adicionamos el número 5
>>> len(Arr)          #Verificamos su tamaño
1
>>> Arr.append(9)
>>> Arr.append(7)
>>> len(Arr)
3
>>> Arr
[5, 9, 7]
>>> Arr.append('hola') #Podemos tener diferentes tipos de dato en el vector
>>> Arr
[5, 9, 7, 'hola']

#También podemos inicializar el vector con 'n' elementos de un valor
>>> val = 3
>>> n = 9
>>> v1 = [val]*n       #Creamos el vector
>>> v1
[3, 3, 3, 3, 3, 3, 3, 3, 3]
>>> len(v1)
9
>>> v1[4] = 7          #Accedemos a la posición 4 index 0
>>> v1                 #para modificar su valor
[3, 3, 3, 3, 7, 3, 3, 3, 3]

#Un uso interesante podría ser para leer un conjunto de números en una línea
>>> v2 = [int(x) for x in input().split()] #split divide en base a un caracter
5 6 7 1 2 4 3 9 8 2 11                  #iteramos sobre la lista de entrada
>>> v2                                     #y casteamos los valores a enteros
[5, 6, 7, 1, 2, 4, 3, 9, 8, 2, 11]
>>> len(v2)
11

#Los bucles en python en realidad hacen iteraciones elemento por elemento en vectores
#llamado también foreach.
>>> for elemento in Arr:
...     print(elemento)
...
5
```

```

9
7
hola
#Aquí elemento contiene el íesimo elemento del vector Arr.

>>> Arr2 = [6, 5, 'a', 'abc']
>>> Arr3 = [1, 2, 3, 4, 5]
>>> len(Arr)
4
>>> len(Arr2)
4
>>> len(Arr3)
5
>>> for el1, el2, el3 in zip(Arr, Arr2, Arr3):    #Podemos iterar multiples
...     print(el1, el2, el3)                    #vectores hasta el más
...                                              #pequeño en un solo for
5 6 1
9 5 2
7 a 3
hola abc 4

#También podemos enumerar la iteración del vector
>>> for i, el in enumerate(Arr):
...     print(i, el)
...
0 5
1 9
2 7
3 hola

```

### 1.6.2. Tuplas

Son muy similares a los vectores, la única e importante diferencia es que una vez creada, la tupla es inmutable, no se puede agregar ni quitar elementos. Una tupla se define con el operador ( )

```

>>> t = (1, 2, 3, 4)    #Tamaño definido
>>> t
(1, 2, 3, 4)
>>> t[0] = 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment

#Al ser inmutable, no podemos editar el
#elemento de la tupla

```

## 1.7. Módulos y Librerías

Python tiene la gran ventaja de contar con una comunidad que provee de librerías open source muy útiles para el desarrollo de proyectos.

Para importar alguna librería o módulo, es necesario usar la instrucción **import**, sin embargo, algunas librerías tienen nombres muy largos y queremos acortarlas, entonces podemos usar la instrucción **as**, al igual que podemos importar submódulos de los módulos originales.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

## Capítulo 2

# Aplicaciones en la Materia

**Descripción:** En este capítulo se presentarán soluciones a ejercicios de prácticas anteriores y si es posible exámenes pasados

### 2.1. Primer Parcial

#### 2.1.1. Algoritmos Secuenciales

Resolver el área de un rectángulo y de un triángulo.

| Entrada                | Salida |
|------------------------|--------|
| Caso rectángulo<br>2 4 | 8      |
| Caso triangulo<br>2 4  | 4      |

Solución

```
#Por defecto en Python leemos una cadena
#usamos split para que cada variable agarre
#un dato separado por un espacio
base, altura = input().split()
#tenemos que convertir a entero cada dato
base, altura = int(base), int(altura)

#Para el caso rectángulo
area_rectangulo = base * altura
print(area_rectangulo)

#Para el caso triangulo
area_triangulo = (base * altura) / 2
print(area_triangulo)
```

Realice un algoritmo que a partir de proporcionarle la velocidad de un automóvil, expresada en kilómetros por hora, proporcione la velocidad en metros por segundo.

| Entrada | Salida |
|---------|--------|
| 36      | 10     |

```
#1km/h * 1000m/1km = 1000m/h * 1h/3600s = 1000m/3600s
velocidad_kmh = int(input())

velocidad_mh = velocidad_kmh * (1000 / 3600)

print(velocidad_mh)
```

### 2.1.2. Estructuras Condicionales

Carlita una niña de 7 años, está aprendiendo las fechas, por lo que su mamá supervisa su aprendizaje. Su madre escribe una fecha valida, Carlita tiene que escribir la siguiente fecha. Resuelva el problema.

| Entrada    | Salida    |
|------------|-----------|
| 27/2/2000  | 28/2/2000 |
| 31/1/2010  | 1/2/2010  |
| 31/12/2010 | 1/1/2011  |
| 30/4/2011  | 1/5/2011  |

```
enero, febrero, marzo, abril, mayo, junio, julio, agosto,\
septiembre, octubre, noviembre, diciembre = 31, 28, 31, 30,\
                                              31, 30, 31, 31,\
                                              30, 31, 30, 31
```

```
#Separamos por el caracter '/'
dia, mes, anho = input().split('/')
dia, mes, anho = int(dia), int(mes), int(anho)
dias = 0
if mes == 1:
    dias = enero
elif mes == 2:
    dias = febrero
elif mes == 3:
    dias = marzo
elif mes == 4:
    dias = abril
elif mes == 5:
    dias = mayo
elif mes == 6:
    dias = junio
elif mes == 7:
    dias = julio
elif mes == 8:
    dias = agosto
elif mes == 9:
    dias = septiembre
elif mes == 10:
    dias = octubre
```

```

elif mes == 11:
    dias = noviembre
elif mes == 12:
    dias = diciembre

dia = dia + 1
if dia <= dias:
    print(str(dia) + '/' + str(mes) + '/' + str(anho))
elif dia > dias:
    dia = dia % dias
    mes = mes + 1
    if mes <= 12:
        print(str(dia) + '/' + str(mes) + '/' + str(anho))
    else:
        print(str(dia) + '/' + '1' + '/' + str(anho+1))

```

Introducir dos números enteros positivos en las variables a y b respectivamente. Calcular y desplegar la cantidad de números pares que existen en el intervalo cerrado entre a y b.

| Entrada | Salida |
|---------|--------|
| 1 11    | 5      |

```

#Leemos varios numeros en una sola linea
a, b = [int(x) for x in input().split()]

contador = 0
for i in range(a, b + 1):
    if i % 2 == 0:
        contador += 1

print(contador)

```

### 2.1.3. Series

Generar la Serie de fibonacci triple: 1, 1, 1, 3, 5, 9, 17, 31, 57...

| Entrada | Salida         |
|---------|----------------|
| 5       | 1, 1, 1, 3, 5, |

```

n = int(input())

a, b, c = 1, 1, -1
for i in range(n):
    d = a + b + c
    a, b, c = b, c, d
    print(str(d) + ', ', end='')

```

Generar la serie compuesta formada por los números fibonacci, números primos y números impares, en ese orden: 1, 2, 1, 1, 3, 3, 2, 5, 5, 3, 7, 7, 5, 11, 9, 8, 13, 11....

| Entrada | Salida                        |
|---------|-------------------------------|
| 10      | 1, 2, 1, 1, 3, 3, 2, 5, 5, 3, |

```

fa, fb = 1, 0 #Fibo
p = 2 #Primos
imp = 1 #Impares

n = int(input())
sw = 0
for i in range(n):
    if sw == 0: #fibonacci
        fc = fa + fb
        fa, fb = fb, fc
        print(str(fc) + ', ', end='')
    elif sw == 1: #primos
        print(str(p) + ', ', end='')
        test = False
        p_temp = p
        while test == False:
            p_temp = p_temp + 1
            p_test = True
            for i in range(2, p_temp):
                if p_temp % i == 0:
                    p_test = False
                    break
            if p_test == True:
                test = True
                p = p_temp
    elif sw == 2: #impares
        print(str(imp) + ', ', end='')
        imp += 2
    sw = (sw + 1) % 3

```

#### 2.1.4. Sumatorias

Dada una variable de entrada x y n, hallar la siguiente sumatoria:

$$S = 1 - \frac{x}{2!} + \frac{x^3}{4!} - \frac{x^5}{6!} + \frac{x^7}{8!}$$

```

n, x = [int(e) for e in input().split()]
n -= 1
S, par, imp, signo = 1, 2, 1, -1
for i in range(n):
    fac = 1
    for i in range(1, par+1):
        fac *= i
    S += (signo * (x**par) / fac)
    par += 2
    imp += 2
    if signo == -1:
        signo = 1
    else:
        signo = -1

```



```
print(S)
```

### 2.1.5. Lotes

Se leen 30 valores enteros (comprendidos entre 5 y 40), que representan la temperatura máxima de cada una de los días de un mes. Se pide hallar e informar: La temperatura máxima del mes y el día que se produjo, cuantos días la temperatura supero los 25°C, El promedio de las temperaturas máximas del mes.

```
pos, max = 0, -(1 << 30)
cont_25, promedio = 0, 0
i = 0
for i, x in enumerate(input().split()):
    x = int(x)
    if x > max:
        pos, max = i, x
    if x > 25:
        cont_25 += 1
    promedio += x
print('Temp Max:', max, 'Día:', pos+1)
print('Días que supera los 25:', cont_25)
print(promedio/(i+1))
```

### 2.1.6. Composición y Descomposición

Dado un número n, eliminar sus dígitos que se encuentren detrás de un 3 o 7.

| Entrada   | Salida  |
|-----------|---------|
| 123456789 | 1235679 |
| 53243247  | 534247  |

```
n = int(input())
nn = 0
while int(n) > 0:
    nn = nn * 10 + (n % 10)
    n = int(n / 10)
prev = -1
while int(nn) > 0:
    e = nn % 10
    if prev != 3 and prev != 7:
        n = n * 10 + e
    nn = int(nn / 10)
    prev = e
print(n)
```

## 2.2. Segundo Parcial

### 2.2.1. Funciones y Procedimientos

Los números primos son aquellos que son solo divisibles por si mismo o el uno. La lista de los primeros números primos es 2,3,5,7,11,13,17,19.

Dados  $0 \leq a, b \leq 10^7$  contar cuantos primos hay en ese rango.

## Entrada

La entrada consiste de varios casos de prueba. La primera línea contiene el número de casos de prueba. Las líneas siguientes corresponden a los casos de prueba, cada caso de prueba contiene dos enteros a,b, La entrada termina cuando no hay más datos.

## Salida

Para cada caso de prueba su programa debe mostrar, en la salida, una línea con el número de primos existente entre a y b inclusivo.

| Entrada | Salida |
|---------|--------|
| 1       |        |
| 2 1000  | 168    |

#Solución no óptima, la aproximación fuerza bruta

```
def primo(n):
    if n == 0 or n == 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

if __name__ == '__main__':
    n = int(input())
    for t in range(n):
        a, b = [int(x) for x in input().split()]
        cont = 0
        for i in range(a, b+1):
            if primo(i):
                cont += 1
        print(cont)
```

Astro tiene una tarea muy importante, de ello depende su nota, su tarea es cambiar los números de base 10 a base X ( $2 \leq X \leq 9$ ), Astro se fue de paseo anoche, te sientes muy culpable por sacarlo a pasear y te sientes culpable por eso decidiste ayudarlo. Para resolver el problema necesitas una función que reciba dos parámetros de entrada cambio\_base(int numero, int nueva\_base).

## Entrada

La entrada consiste de varios casos de prueba.

## Salida

Para cada caso de prueba su programa debe mostrar, en la salida, una línea con el número en base X.

| Entrada             | Salida           |
|---------------------|------------------|
| 1 2                 | 1                |
| 10 2                | 2                |
| 100 3               | 9                |
| 123 4               | 27               |
| 1234 6              | 310              |
| 12345678876543218 9 | 2345228615981168 |

```
import sys

def cambio_base(num, base):
    print(int(str(num), base))

if __name__ == '__main__':
    for line in sys.stdin:
        n, b = [int(x) for x in line.split()]
        cambio_base(n, b)
```

### 2.2.2. Vectores

#### Recorrido

Escribir un programa que lea una secuencia de números naturales, y los imprima en orden inverso.

#### Entrada

La entrada consiste de múltiples secuencias. Cada secuencia comienza con un número natural que indica la longitud de la secuencia. Termina cuando no hay más secuencias.

#### Salida

Imprima, línea por línea, las secuencias en el orden inverso.

| Entrada             | Salida            |
|---------------------|-------------------|
| 6 10 11 12 15 14 13 | 13 14 15 12 11 10 |
| 0                   |                   |
| 2 22 21             | 21 22             |

```
"""
En este caso, se da el número de elementos como entrada
Sin embargo en Python, al poder leer una línea entera, no
es necesario saber el tamaño para convertirlo todo a un vector
así que leeremos el vector en la entrada, y luego eliminaremos
el primer elemento
"""

entrada = [int(x) for x in input().split()]
#Eliminamos el primer elemento del vector mediante este operador
#Podemos eliminar n primeros elementos con [n:]
v = entrada[1:]

#Ahora le daremos la vuelta al vector, recorriendo inversamente
"""
Podemos iterar de dos formas diferentes, en la forma clásica de C
mediante una variable iteradora para acceder al i-ésimo elemento
for i in range(n):
    v[i]
Sin embargo esa es una forma de código no Pythonica, siendo la
forma recomendada, hacerlo como un foreach
"""
for elemento in reversed(v): #Usamos reversed para darle la vuelta
    print(elemento, end=' ')
```

```
print('\n—————')
```

#Sin embargo la forma más corta con Python es:

#El operador \* para listas, desempaqueta todos sus elementos

```
print( *reversed(v) )
```

## Criba de Eratóstenes

La criba de Eratóstenes es un algoritmo el cual consiste en tachar todos los múltiplos de un número que se sabe que es primo, de esta forma, empezamos por el 2, y tachamos todos los pares, entonces avanzamos al siguiente elemento no tachado, el 3, y se tachan los múltiplos de 3, luego se avanza al 5 porque el 4 ya está tachado, y así sucesivamente.

Por fines de optimización, se corre el primer bucle sólo hasta la raíz de n, ya que se sabe que hasta ahí existirá como máximo un primo, al igual que empezamos a tachar más allá de su cuadrado perfecto.

```
def criba(n):
    #n: Nro de elementos a cribar
    #Creamos un vector de verdades de tamaño n
    primos = [True]*(n+1)
    #Asignamos False a los valores 0 y 1 porque no son primos
    primos[:2] = [False]*2
    i = 2
    while i*i <= n:
        if primos[i] == True:
            j = i*i
            while j <= n:
                primos[j] = False
                j += i
            i += 1
    return primos
```

De esta forma podemos resolver nuevamente el ejercicio de la sección 2,2,1 de una forma mucho más óptima y para casos mucho más grande.

Para los siguientes casos:

| Entrada          | Salida |
|------------------|--------|
| 4                |        |
| 2 1000           | 168    |
| 100000 1000000   | 68906  |
| 1000000 10000000 | 586081 |
| 2 10000000       | 664579 |

```
def criba_acumulada(n):
    """
    Podemos re escribir la criba para que nos devuelva un vector
    que cuente la cantidad de primos acumulados en rangos.
    """
    primos = [True]*(n+1)
    contador = [0]*(n+1)

    primos[:2] = [False]*2
    for i in range(2, n+1):
        if primos[i] == True:
```

```

        contador[i] = contador[i-1] + 1
        j = i*i
        while j <= n:
            primos[j] = False
            j += i
        else:
            contador[i] = contador[i-1]
    return contador

if __name__ == '__main__':
    n = int(input())
    #Pre calculamos la cantidad de primos en una sola
    #ida de 10^7
    n_primos = criba_acumulada(100000000)
    for i in range(n):
        a, b = [int(x) for x in input().split()]
        print(n_primos[b] - n_primos[a-1])

```

### 2.2.3. Cadenas

Las cadenas son internamente listas de caracteres, teniendo las mismas operaciones que los vectores, y algunos extras para manipulación de texto.

Fiona siempre amó la poesía, y recientemente descubrió una forma poética fascinante. Tautogramas, son un caso especial de aliteración, que es la ocurrencia de la misma letra en el inicio de palabras adyacentes. En particular, una oración es un tautograma si todas sus palabras comienzan con la misma letra. Ayude a fiona a saber si las oraciones que escribió son tautogramas o no.

#### Entrada

Cada caso de entrada tendrá una oración.

#### Salida

Por cada oración imprimir Y si es un tautograma y N si no lo es.

| Entrada                      | Salida |
|------------------------------|--------|
| Flowers Flourish from France | Y      |
| Sam Simmonds speaks softly   | Y      |
| Peter pIckEd pePPers         | Y      |
| truly tautograms triumph     | Y      |
| this is NOT a tautogram      | N      |

```

def es_tautograma(cads):
    """
    :param list[str] cads:
    """
    #Primer caracter de la primera cadena
    caracter = cads[0][0].lower()
    #La función lower convierte el caracter en minúsculas
    for cad in cads[1:]: #Iteramos desde el segundo elemento
        #Comparamos los primeros caracteres
        if caracter != cad[0].lower():
            return False
    return True

```

```

if __name__ == '__main__':
    #Leemos la cadena y la separamos por espacios
    palabras = input().split()
    print( 'Y' if es_tautograma(palabras) else 'N' )

```

---

#### 2.2.4. Recursividad

La recursividad es un paradigma, una forma de afrontar problemas mediante definiciones recurrentes, es decir, definiendo las transiciones de estado como llamadas a la misma función con diferentes argumentos, hasta que llegue a cumplirse la o las condiciones iniciales, una vez cumplidas, se finaliza la recursión y devuelve un valor final.

Un ejemplo muy clásico es la solución del Fibonacci Recursivo.

```

def fibo(n):
    #Casos base, casos en los que sabemos la verdad
    #Si me piden el elemento 0 de la serie retorno 0
    if n == 0: return 0
    #Si me piden el elemento 1 de la serie retorno 1
    if n == 1: return 1

    #Retorno la suma según la definición recursiva de
    #la serie fibonacci
    return fibo(n-1) + fibo(n-2)

if __name__ == '__main__':
    for i in range(10):
        print(fibo(i))

```

---

Otro ejemplo muy famoso que aplica recursividad, es el ordenamiento óptimo, como ser el Merge sort:

```

def merge(p, q, r, A):
    n1, n2 = q - (p - 1), r - q
    a = [0]*(n1+1)
    b = [0]*(n2+1)
    #Máximo número entero posible
    a[n1] = b[n2] = (1<<30)
    for i in range(n1):
        a[i] = A[p + i]

    for i in range(n2):
        b[i] = A[(q + 1) + i]

    i = j = 0
    for k in range(p, r+1):
        if a[i] <= b[j]:
            A[k] = a[i]
            i += 1
        else:
            A[k] = b[j]
            j += 1

```

```
def sort(p, r, A):
    if p < r:
        q = int( (p+r)/2 )
        sort(p, q, A)
        sort(q+1, r, A)
        merge(p, q, r, A)

if __name__ == '__main__':
    A = [int(x) for x in input().split()]
    sort(0, len(A)-1, A)
    print(A)
```

---

## 2.3. Final

### 2.3.1. Matrices

Por definición, se convierten en vectores n-dimensionales, no habría mucho que desarrollar en cuanto a su uso más que introducción a notación.

---

```
n = 5
m = 6

#n x m
mat = [[0 for j in range(m)] for i in range(n)]
#Así podemos crear una matriz de tamaño nxm y la
#rellenamos de ceros.

#Para acceder a un elemento se indexa en cada dimensión
#mat[alto, ancho, profundidad, etc]
```

---

## Capítulo 3

# Ejercicios Generales

### 1. Pig Latin

Es un lenguaje que toma la primera letra de una palabra, la pone al final y le concatena “ay”. La única excepción ocurre cuando la primera letra es una vocal, en cuyo caso la mantenemos como tal y le concatenamos “hay” al final.

Ej: boot  $\longrightarrow$  ootbay, image  $\longrightarrow$  imagehay

Escribe una función que reciba como argumento una palabra y la convierta a Pig Latin.

### 2. List Comprehensions

Es una estructura muy útil que deriva de los constructores del cálculo lambda, nos permite facilitar mucho el código.

Ej: `[int(x) for x in input().split()]`

- Escriba una List Comprehension que imprima por pantalla una lista de los números cúbicos del 1 al 10
- Escriba una List Comprehension que imprima los posibles resultados de dos lanzamientos de moneda. (Pueden usarse estructuras anidadas).
- Escriba una función que reciba una cadena, y con una List Comprehension retorne todas las vocales de la cadena.

### 3. Oraciones en Pig Latin

Resuelva nuevamente el ejercicio 1, pero extendido al caso de oraciones completas.

### 4. Retos con List Comprehension

- Escriba una función que reciba una lista de elementos de tipos diferentes y use una List Comprehension para retornar los elementos de la lista de tipo int.
- Escriba una List Comprehension que resuelva la ecuación  $y = x^2 + 1$ . Tu solución debería imprimir una lista de pares  $[x, y]$ .  
Use el dominio  $x \in [-5, 5]$  y el rango  $y \in [0, 10]$ .

### 5. Sumas

Escriba un programa que compute la suma de los logaritmos de todos los primos desde 2 hasta  $10^7$ , e imprima la suma de los logaritmos de los primos.



## 6. Sub-Cadenas1

Escriba dos funciones que reciban dos cadenas (a,b), y que cuenten el número de ocurrencias de la palabra “a” en “b”.

## 7. Sub-Cadenas2

Escriba una función que reciba dos argumentos, dos cadenas “a” y “b”. Debe retornar una lista de tuplas con las posiciones de inicio de cada ocurrencia de la cadena “a” en “b”, index 0.

## 8. Sub-Cadenas3

Dados tres argumentos, la posición de inicio de la primera sub cadena, la posición de inicio de la segunda sub cadena, y la longitud de la primera sub cadena. Retorne todos los inicios de las ocurrencias de la primera sub cadena en la segunda sub cadena.

## 9. Intersección de Listas

Mediante la definición de intersección de conjuntos.

([https://es.wikipedia.org/wiki/Intersecci%C3%B3n\\_de\\_conjuntos](https://es.wikipedia.org/wiki/Intersecci%C3%B3n_de_conjuntos))

Escriba una función, que dadas dos listas, devuelva la intersección de estas.

Ej: `list_intersection([1, 3, 5], [5, 3, 1]) = [1, 3, 5]`

## 10. Intersección de balones

Dadas las coordenadas del centro y radio para dos bolas, en forma de tuple (x1, y1, r1), responda si los balones se intersectan o no.

## 11. Palíndromes

Dada una cadena, verifique si es palíndrome. (Se lee igual de izquierda a derecha, como de derecha izquierda.)

## 12. Fi de Euler

Aplicando la criba de eratóstenes y un análisis de la definición, escriba el código más óptimo que pueda para evaluar la función Fi de Euler.

[https://es.wikipedia.org/wiki/Funci%C3%B3n\\_%CF%86\\_de\\_Euler](https://es.wikipedia.org/wiki/Funci%C3%B3n_%CF%86_de_Euler)

## 13. Cifrado

Dado un algoritmo de cifrado que cumple las siguientes reglas:

- Si la longitud de la cadena a cifrar es de 1 o 2, entonces retornamos la misma cadena
- Si “S”, es una cadena de “N”, caracteres, y  $k = \lfloor N/2 \rfloor$  entonces:

$$\text{enc}(S) = \text{encrypt}(s_k s_{k-1} \dots s_2 s_1) + \text{encrypt}(s_N s_{N-1} \dots s_{k+1})$$

Implementelo.

## 14. Producto Matricial

Dadas dos Matrices A y B, calcule su producto matricial.

[https://es.wikipedia.org/wiki/Multiplicaci%C3%B3n\\_de\\_matrices](https://es.wikipedia.org/wiki/Multiplicaci%C3%B3n_de_matrices).

### 15. Cuadrado mágico

Dada una matriz, verifique que se aun cuadrado mágico.

# Bibliografía

## Cursos OCW MIT

- [https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-092-introduction-to-programming-in-java-january-iap-2010/assignments/MIT6\\_092IAP10\\_assn07.pdf](https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-092-introduction-to-programming-in-java-january-iap-2010/assignments/MIT6_092IAP10_assn07.pdf)
- <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-092-introduction-to-programming-in-java-january-iap-2010/assignments/>
- <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-087-practical-programming-in-c-january-iap-2010/assignments/>
- <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-096-introduction-to-c-january-iap-2011/assignments/>
- <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-s096-effective-programming-in-c-and-c-january-iap-2014/assignments/>
- <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/assignments/>
- <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-00-introduction-to-computer-science-and-programming-fall-2008/assignments/>
- <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-189-a-gentle-introduction-to-programming-using-python-january-iap-2011/assignments/>

## Libro de Python por Guido van Rossum

- <https://argentinaenpython.com/quiero-aprender-python/TutorialPython3.pdf>